Hacettepe University

Computer Engineering Department

BBM479/480 End of Project Report

Project Details

Title	Autonomous Navigation for Duckiebots
Supervisor	Özgür Erkent

Group Members

	Full Name	**Student ID**
1	Ömer Kayra Çetin	2210356060
2	Yunus Emre Uluer	2210356102
3	Mehmet Yılmaz Erdem	2210765036
4	Mustafa Said Oğuztürk	2210765027
5	Arda Deniz Ayyıldız	2210765018

Abstract of the Project (/ 10 Points)

Explain the whole project shortly including the introduction of the field, the problem statement, your proposed solution and the methods you applied, your results and their discussion, expected impact and possible future directions. The abstract should be between 250-500 words.

This project develops a complete, full-stack autonomous navigation system for the resource-constrained Duckiebot platform. Achieving advanced autonomy on low-cost mobile robots presents significant engineering challenges, as embedded systems typically lack the onboard computational power required to process complex perception and planning algorithms in real-time. Initial testing revealed that running these high-level tasks locally pushed the Duckiebot's CPU load to nearly 98%, bottlenecking performance and causing system instability. To overcome these strict hardware limitations and achieve our primary navigation goals, this project proposes and implements a highly optimized distributed system architecture. By utilizing a LAN-optimized Robot Operating System (ROS) network, computationally intensive modules are strategically offloaded to a remote workstation. This execution split ensures approximately 10ms communication latency, allowing the Duckiebot to maintain lightweight, real-time closed-loop stability onboard while benefiting from powerful remote processing.

Enabled by this distributed architecture, the core of the system features two distinct but complementary autonomous navigation modules. The first module is designed for continuous, reactive trajectory alignment. It utilizes a vision-based lane-following system driven by a PID controller, augmented by a fine-tuned YOLOv5 model running remotely for real-time object detection. This module relies on a parameterized maneuver function to successfully navigate around known or static obstacles. The second module is dedicated to complex, global routing and dynamic execution. It employs a hybrid path planner that generates optimal global routes across a tiled cost-map using the A* algorithm, while the Dynamic Window Approach (DWA) facilitates local path execution and dynamic obstacle avoidance. Both navigation modules are supported by a robust localization pipeline that fuses wheel encoder dead-reckoning with periodic ARTag/AprilTag landmark matching to mitigate sensor drift.

Finally, user interaction and system monitoring are seamlessly integrated through a custom cross-platform 3D interface for Web and Mobile. This application provides live telemetry, spatial tracking within the mapped environment, and manual override capabilities via WebSockets. Ultimately, this project successfully demonstrates that advanced, multi-layered autonomous mechanisms can execute reliably on low-cost embedded hardware by harmonizing distributed computing with sophisticated navigation pipelines.

Introduction, Problem Definition & Literature Review (/ 20 Points)

Introduce the field of your project, define your problem (as clearly as possible), review the literature (cite the papers) by explaining the proposed solutions to this problem together with limitations of these problems, lastly write your hypothesis (or research question) and summarize your proposed solution in a paragraph. Please use a scientific language (you may assume the style from the studies you cited in your literature review). You may borrow parts from your previous reports but update them with the information you obtained during the course of the project. This section should be between 750-1500 words.

Introduction to the Field

The field of autonomous mobile robotics has experienced rapid advancement, primarily driven by the deployment of complex perception, mapping, and planning algorithms. High-end autonomous vehicles leverage robust sensor suites including LiDAR, radar, and advanced stereo cameras coupled with immense onboard computing power to navigate dynamic, unstructured environments. However, translating these sophisticated autonomy paradigms to low-cost, resource-constrained embedded systems remains a critical engineering challenge. Platforms utilized for educational and foundational research, such as the differential-drive Duckiebot DB21M operating within the structured urban setting of Duckietown, provide an excellent testbed for this challenge. Success in this domain requires highly optimized software architectures capable of delivering real-time performance without relying on brute-force computational power.

Problem Definition

The central problem addressed in this project is the implementation of a robust, fully autonomous navigation stack on hardware that is severely computationally and sensorily constrained. The Duckiebot platform relies on a basic sensor array: a monocular camera, wheel encoders, and an Inertial Measurement Unit (IMU). These sensors inherently lack depth perception and are highly susceptible to cumulative errors. For example, encoder-based dead-reckoning suffers significantly from wheel slippage and floor irregularities over time.

More critically, the physical onboard processing unit presents a severe bottleneck. Initial development revealed that attempting to execute foundational perception models and high-level control loops locally saturated the robot's CPU at approximately 98% utilization. This computational overload induces severe system latency, making it impossible for the robot to react to dynamic obstacles or maintain closed-loop stability in real-time. Therefore, the problem extends beyond simply writing algorithms; it is an architectural problem of orchestrating complex, multi-layered autonomy (localization, vision, path planning, and obstacle avoidance) on hardware that cannot natively support monolithic execution.

Literature Review

The development of a complete navigation stack requires integrating several distinct robotic disciplines. A review of the literature highlights the gap between theoretical solutions and the realities of constrained edge devices.

- **Lane Following and Control:** Vision-based lane following has been widely explored. Castro et al. [1] demonstrated effective visual feedback systems for lane keeping , and Samuel et al. [2] reviewed robust path-tracking methodologies such as the Pure Pursuit and PID controllers. While these algorithms are computationally lightweight, they are highly sensitive to environmental lighting variations and shadows, which can disrupt the image processing pipelines used to detect lane boundaries.
- **Obstacle Avoidance and Perception:** Pandey et al. [3] provided a comprehensive review of mobile robot navigation and obstacle avoidance techniques. Traditional, highly reliable obstacle avoidance usually requires active sensors, such as LiDAR or Time-of-Flight (ToF) cameras. Modern vision-based alternatives rely on Deep Neural Networks (DNNs) for object detection. However, deploying models like YOLOv5 directly on standard embedded microprocessors results in frame-rate drops and unmanageable latency, delaying critical motor commands.
- **Localization and Sensor Fusion:** Determining the robot's precise pose is the most persistent challenge in mobile robotics. He et al. [4] reviewed monocular visual odometry techniques, while Brossard et al. and Yu & Jiang [5] explored the fusion of wheel odometry with IMU data to create more resilient pose estimates. Advanced frameworks like ORB-SLAM offer Simultaneous Localization and Mapping to correct sensor drift. However, feature-based Monocular SLAM requires substantial memory overhead and processing cycles, ultimately rendering it unviable for the Duckiebot without sacrificing other core navigation functions.
- **Path Planning:** For point-to-point autonomy, Ming et al. [7] surveyed various path-planning algorithms, establishing the effectiveness of hybrid architectures. These typically combine a global routing algorithm (such as A*) to find the shortest path, with a local reactive planner, such as the Dynamic Window Approach (DWA), to handle dynamic execution and localized obstacle clearance. Forward-simulating trajectories in DWA, however, further taxes the CPU.

In summary, the existing literature proposes excellent individual solutions for navigation, but these solutions generally assume the availability of either high-fidelity sensors or substantial onboard computing power, neither of which applies to this problem context.

Hypothesis

We hypothesize that the strict hardware limitations of low-cost embedded robotics can be successfully bypassed through a distributed, LAN-optimized system architecture. By decoupling execution, restricting the onboard hardware to lightweight, low-level reactive loops while offloading perception-heavy and planning-intensive algorithms to a remote workstation, a resource-constrained robot can achieve stable, real-time, multi-layered autonomy without experiencing computational saturation.

Proposed Solution Summary

To validate this hypothesis, we developed a distributed Robot Operating System (ROS) architecture. Heavy computations are offloaded to a remote workstation via Wi-Fi, optimized to maintain a highly responsive communication latency of approximately 10ms. Operating on this architecture, our navigation stack is divided into two distinct, specialized modules. The first module focuses on reactive trajectory execution: it features a PID-driven lane-following system that estimates lateral and heading errors from extracted markings. Operating concurrently, a remote YOLOv5 model handles object detection, actively triggering parameterized maneuvering functions to safely negotiate around known obstacles within the lane. The second module is dedicated to complex spatial routing. It utilizes a hybrid planner where the A* algorithm generates optimal global paths across a predefined tiled cost-map, and the DWA local planner forward-simulates trajectories to safely execute the path while dynamically clearing unforeseen obstacles. To provide stable spatial awareness without the crushing overhead of full SLAM, localization relies on encoder-based dead-reckoning, which is periodically corrected by map-aligned ARTag/AprilTag fiducial markers. Finally, this entire ecosystem is managed through a custom cross-platform 3D Web/Mobile Interface utilizing WebSockets, providing users with live spatial telemetry and manual control overrides.

References

[1] Castro, O., Céspedes, A., Ubaldo, R., & Ramos, O. E. (2019, November). Lane following with a duckiebot vehicle using visual feedback. In *2019 IEEE Sciences and Humanities International Research Conference (SHIRCON)* (pp. 1-4). IEEE.

[2] Samuel, M., Maziah, M., Hussien, M., & Godi, N. Y. (2018). Control of autonomous vehicle using path tracking: A review. *Advanced Science Letters*, *24*(6), 3877-3879.

[3] Pandey, A., Pandey, S., & Parhi, D. (2017). Mobile robot navigation and obstacle avoidance techniques: A review. *Int Rob Auto J*, *2*(3), 00022.

[4] He, M., Zhu, C., Huang, Q., Ren, B., & Liu, J. (2020). A review of monocular visual odometry. *The Visual Computer*, *36*(5), 1053-1065.

[5] Brossard, M., & Bonnabel, S. (2019, May). Learning wheel odometry and IMU errors for localization. In *2019 International Conference on Robotics and Automation (ICRA)* (pp. 291-297). IEEE.

[6] Yu, S., & Jiang, Z. (2020). Design of the navigation system through the fusion of IMU and wheeled encoders. *Computer Communications*, *160*, 730-737.

[7] Ming, Y., Li, Y., Zhang, Z., & Yan, W. (2021). A survey of path planning algorithms for autonomous vehicles. *SAE international journal of commercial vehicles*, *14*(02-14-01-0007), 97-109.

Methodology (/ 25 Points)

Explain the methodology you followed throughout the project in technical terms including datasets, data pre-processing and featurization (if relevant), computational models/algorithms you used or developed, system training/testing (if relevant), principles of model evaluation (not the results). Using equations, flow charts, etc. are encouraged. Use sub-headings for each topic. Please use a scientific language. You may borrow parts from your previous reports but update them with the information you obtained during the course of the project. This section should be between 1000-1500 words (add pages if necessary).

Distributed System Architecture

To circumvent the strict computational constraints of the Duckiebot platform, a distributed architecture was implemented using the Robot Operating System (ROS). The execution pipeline was bifurcated: low-level hardware actuation (motor drivers, encoder polling) remained on the embedded hardware, while computationally intensive tasks (perception, global planning) were offloaded to a remote workstation.

Communication between the robot and the workstation was facilitated via a LAN-optimized WebSocket bridge. This network optimization maintained an average latency of approximately 10ms, ensuring that critical motor commands and sensor telemetry were synchronized in real-time. This architectural shift successfully reduced the onboard CPU load from ~98% to a stable ~35%, providing the necessary headroom for closed-loop stability.

Vision and Perception Pipeline

The perception layer is responsible for extracting navigable features and dynamic obstacles from the monocular camera feed. It is divided into two distinct parallel processes: traditional computer vision for lane tracking and deep learning for object detection.

Lane Detection and Extraction

The lane-following module utilizes the OpenCV library for real-time image processing. The raw RGB camera stream is first converted into the HSV (Hue, Saturation, Value) color space, which is significantly more robust to ambient lighting variations than the standard RGB space. Adaptive thresholds are applied to isolate yellow and white lane markings.

Once the color masks are applied, a Canny edge detector highlights the boundaries of the lane markings, and a Probabilistic Hough Transform extracts linear segments. These segments are passed into a Histogram Filter (Lane Filter) to estimate the robot's lateral offset d and heading error ϕ relative to the center of the lane.

Object Detection and Persistent Obstacle Memory
To detect obstacles, specifically "Duckies" and other "Duckiebots", a YOLOv5s object detection model was fine-tuned and executed on the remote workstation. However, relying purely on instantaneous visual detections is insufficient due to the camera's limited field of view (FOV).

To solve this, an obstacle memory mechanism was developed to decouple obstacle detection from planning. When YOLOv5s outputs a bounding box, it projects the detection onto the ground to establish a local position within the robot's frame. Utilizing ROS TF2, this local position is transformed into global map coordinates. The package maintains a persistent state for each obstacle, assigning semantic priors (physical radius and safety margin) based on the object's class. A moving average temporal filter smooths the obstacle's position over consecutive frames. Crucially, the package manages the obstacle's lifecycle; if an obstacle leaves the camera's FOV, the system retains its last known global position until a maximum age timeout is reached, allowing the planner to safely navigate around "remembered" obstacles.

Pose Estimation and Localization

Given the decision to remove highly taxing Monocular SLAM from the pipeline, the system required a lightweight yet accurate method for spatial tracking. This was achieved by combining kinematic dead-reckoning with visual fiducial marker correction.

Encoder-Based Dead-Reckoning
Initial high-frequency pose estimation is calculated using onboard wheel encoders. By measuring the rotation of the left and right wheels, the robot's linear velocity v and angular velocity ω are derived using the differential drive kinematic model:

$$v = \frac{R}{2}(\omega_r + \omega_l)$$

$$\omega = \frac{R}{L}(\omega_r - \omega_l)$$

Where R is the wheel radius, L is the wheelbase, and ω_r, ω_l are the angular velocities of the right and left wheels, respectively. The robot's global pose (x, y, θ) is then propagated over discrete time steps Δt:

$$x_{t+1} = x_t + v \cdot \cos(\theta_t) \cdot \Delta t$$

$$y_{t+1} = y_t + v \cdot \sin(\theta_t) \cdot \Delta t$$

$$\theta_{t+1} = \theta_t + \omega \cdot \Delta t$$

Drift Mitigation via ARTag Landmarks

Because dead-reckoning is highly susceptible to cumulative drift due to wheel slippage, periodic absolute position updates are required. ARTag/AprilTag fiducial markers are placed at known strategic locations within the Duckietown map. When the monocular camera detects a tag, the visual pipeline computes the $SE(3)$ transformation matrix from the camera to the tag. Since the tag's global coordinates are known, this transform provides an absolute measurement of the robot's pose, instantly overwriting accumulated drift in the odometry frame.

Reactive Control and Safety Navigation

The first of the two primary navigation modules are designed for immediate, reactive maneuvering, ensuring the robot remains within safe operational bounds while following lanes.

PID Steering Control

To maintain continuous trajectory alignment within a lane, a Proportional-Integral-Derivative (PID) controller calculates a steering command (angular velocity, ω) based on the lateral and heading errors estimated by the Lane Filter:

$$\omega(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

Where $e(t)$ is a weighted combination of the lateral and heading errors, and K_p, K_i, K_d are the empirically tuned controller gains.

Safety Deceleration and Evasive Maneuvers

When the YOLOv5s model detects an object in the robot's direct path, a safety module dictates the robot's longitudinal velocity. To prevent abrupt braking, a smooth linear deceleration profile is applied when an object crosses a predefined safety threshold. For static obstacles within the lane, a parameterized maneuver function temporarily overrides the PID controller, executing a predetermined trajectory to navigate around the known object before resuming lane tracking.

Global and Local Path Planning

The second navigation module handles complex, goal-oriented autonomy. This hierarchical planning architecture operates on a predefined static map to guarantee stable point-to-point navigation.

Cost-Map Generation & Global Routing (A* Algorithm)

The navigation graph is constructed by discretizing Duckietown's tiled road map into a directed graph. Nodes are extracted from intersection tiles (turns, 3-way junctions, 4-way intersections) and straight segments. Between adjacent intersection nodes, intermediate waypoints are generated using Bezier curves for turns and linear interpolation for straight segments, enabling smooth trajectory generation. Edge weights are assigned as Manhattan distances between consecutive nodes in the tile coordinate system.

To compute an optimal route to a user-defined goal, the A* algorithm explores nodes using the cost function $f(n) = g(n) + h(n)$, where $g(n)$ is the accumulated Manhattan distance from the start node and $h(n)$ is the Euclidean distance heuristic to the goal node. This combination of heuristic guidance with actual traversal costs enables efficient discovery of globally optimal paths through the discretized road network.

Custom Local Execution (Dynamic Window Approach)
To execute the A* path while dynamically avoiding obstacles, a custom custom DWA planner was developed. Standard ROS costmap_2d implementations are exceptionally resource-heavy; thus, this custom DWA planner provides lightweight, high-frequency (10 Hz) local planning tailored for the Duckiebot's constraints.

The DWA operates by sampling a discrete window of linear and angular velocity pairs (v, ω). This dynamic window is constrained by the robot's current velocity and its maximum kinematic acceleration limits, ensuring all sampled velocities are physically reachable. The planner forward-simulates short trajectories (prediction horizon) for each velocity pair using differential drive kinematics.

Each simulated trajectory is evaluated against a multi-objective cost function to select the most optimal command. The total cost is calculated as:

$$C_{total} = W_{path}C_{path} + W_{obs}C_{obs} + W_{vel}R_{vel} + W_{head}C_{head}$$

- **Path Cost (C_{path}):** Measures the geometric deviation from the A* global path to enforce lane following.
- **Obstacle Cost (C_{obs}):** Evaluates distance to the global coordinates provided by the obstacle memory. It applies a repulsive potential field, returning an infinite cost penalty to strictly reject trajectories that result in a collision within an object's safety margin.
- **Velocity Reward (R_{vel}):** A negative cost that incentivizes higher safe speeds to ensure forward progress.
- **Heading Cost (C_{head}):** Evaluates how well the trajectory's final angle aligns with the global path, ensuring smooth turning behaviors.

The trajectory yielding the lowest C_{total} is selected, and its corresponding (v, ω) command is published to the motor drivers.

Human-Machine Interface (HMI)

To monitor and control the distributed system, a custom cross-platform (Web and Mobile) interface was developed. Relying on mDNS for local network discovery, the frontend leverages Three.js to render a hardware-accelerated 3D environment. Through the WebSocket bridge, the application receives high-frequency telemetry (pose, velocity, YOLOv5 bounding boxes, camera feed). This allows operators to view live spatial tracking of the robot, input target coordinates for the A* planner, or seize manual control via virtual joysticks in real-time.

Results & Discussion (/ 30 Points)

Explain your results in detail including system/model train/validation/optimization analysis, performance evaluation and comparison with the state-of-the-art (if relevant), ablation study (if relevant), a use-case analysis or the demo of the product (if relevant), and additional points related to your project. Also include the discussion of each piece of result (i.e., what would be the reason behind obtaining this outcome, what is the meaning of this result, etc.). Include figures and tables to summarize quantitative results. Use sub-headings for each topic. This section should be between 1000-2000 words (add pages if necessary).

System-Level Architecture & Performance Evaluation

The primary and most impactful result of this project was the successful validation of the distributed ROS architecture. Early in the development cycle, a critical hardware bottleneck was identified: the Duckiebot's embedded on-board processing unit was incapable of simultaneously handling low-level motor control, visual perception, and complex path planning.

Architectural Ablation and CPU Optimization

During initial monolithic testing where all nodes including the YOLOv5s model and planning algorithms were executed locally on the Duckiebot, the system's CPU utilization consistently saturated at approximately 98%. This extreme computational load resulted in severe thermal throttling, frame-rate drops in the camera pipeline, and critical delays in publishing motor commands, ultimately rendering the system unsafe and unstable.

By implementing the distributed architecture, which acts as a practical ablation of on-board vs. off-board computing, heavy computational tasks were offloaded to a remote workstation. This execution split successfully and dramatically reduced the Duckiebot's on-board CPU load to a stable ~35%.

Discussion: The implication of this 63% reduction in CPU load is profound. It demonstrates that resource-constrained embedded platforms do not need to be physically upgraded to achieve state-of-the-art autonomy, provided the software architecture is sufficiently optimized for networked execution. To ensure this distributed setup did not introduce hazardous control delays, a LAN-optimized WebSocket bridge was established, achieving an average communication latency of ~10ms. This minimal latency proved entirely sufficient to maintain real-time closed-loop stability, effectively proving our core hypothesis.

Navigation Module 1: Reactive Control Performance

The first navigation module, responsible for immediate trajectory alignment and safety maneuvering, was evaluated based on its qualitative stability and responsiveness in the test environment.

Lane Following Stability and Environmental Sensitivity

The visual lane-following system, utilizing OpenCV in the HSV color space, successfully maintained the robot within the lane boundaries during continuous operation. The PID controller effectively minimized lateral and heading errors, resulting in smooth, non-oscillatory steering.

However, testing revealed a notable sensitivity to environmental conditions. The vision-based system is highly susceptible to changes in ambient lighting and the presence of shadows.

Discussion: Shadows casting harsh lines across the track occasionally caused the Canny edge detector to misinterpret lighting gradients as physical lane markings. While adaptive thresholding mitigated this to a degree, it highlights a fundamental limitation of purely vision-based traditional control. This result justifies future work into dynamic exposure control or supplementary sensor fusion to guarantee robustness under variable lighting.

Object Detection and Obstacle Memory Efficacy
The integration of the fine-tuned YOLOv5s model yielded highly accurate real-time classifications of "Duckies" and "Duckiebots". More importantly, the introduction of the custom obstacle memory package completely transformed the robot's reactive capabilities. By decoupling the instantaneous camera FOV from the robot's spatial awareness, the system effectively maintained a "memory" of obstacles.

Discussion: When an obstacle exited the camera frame during a tight turn, the temporal moving-average filter and global map projection allowed the safety deceleration logic to continue operating seamlessly. The linear deceleration profile functioned exactly as designed, bringing the Duckiebot to a smooth, controlled halt at the predefined critical distance, preventing abrupt stops that could induce wheel slip.

Navigation Module 2: Planning and Dynamic Execution

The performance of the hierarchical planning architecture was evaluated based on its ability to generate collision-free routes and execute them dynamically.

Global Routing (A*) on Tiled Cost-Maps
The strategic decision to utilize a static, predefined directed graph constructed from Duckietown's tiled road segments, rather than a dynamically generated SLAM map, proved highly effective for the structured environment. The A* algorithm efficiently generates globally optimal trajectories from arbitrary start positions to user-defined goals, with path validity enforced through directed edge constraints that respect lane directionality and intersection connectivity. Intermediate waypoints (generated via Bezier curves for turns and linear interpolation for straight segments) enable smooth motion between intersection nodes.

Discussion: This approach provides a highly stable, verifiable, and computationally efficient global reference path. In a structured "Live City" concept, relying on a pristine, predefined topological graph is more reliable than dynamically constructing one from sensor data, particularly when sensor noise is high or when real-time SLAM is impractical. The graph-based abstraction decouples the discrete navigation decisions (which intersection to traverse) from continuous control, allowing the local planner (DWA) to focus on dynamic obstacle avoidance without replanning the entire route. This separation of concerns significantly reduces computational overhead while maintaining path optimality within the predefined network topology.

Dynamic Window Approach (DWA) Feasibility
The custom, lightweight DWA node successfully bridged the gap between the static global path and the dynamic physical environment. Operating at a high frequency of 10 Hz, the DWA algorithm consistently sampled kinematically feasible velocities.

Discussion: The tuning of the DWA cost function weights ($W_{path}, W_{obs}, W_{vel}, W_{head}$) was critical to the observed results. Placing a disproportionately high penalty (infinite cost) on the obstacle proximity score ensured that the robot strictly prioritized collision avoidance over trajectory adherence. When a YOLOv5s detection was projected into the map by the memory node, the DWA naturally generated a smooth, curving velocity profile that deviated from the A* path just enough to clear the object's safety margin, before the heading and path costs gently guided the robot back to the lane center.

Localization, Odometry, and the SLAM Trade-off

A major pivot in the project's development lifecycle was the removal of Visual Odometry and Monocular SLAM from the proposed objectives. The results of our localization pipeline directly justify this architectural trade-off.

Encoder Drift and ARTag Mitigation
As anticipated during the risk assessment phase, relying exclusively on wheel encoder dead-reckoning resulted in significant spatial drift over time due to wheel slippage and microscopic floor irregularities. Without correction, the internal odometry frame would eventually diverge entirely from the global map frame, causing the DWA planner to execute maneuvers based on hallucinated coordinates.

To combat this, the system utilized map-aligned ARTag/AprilTag fiducial markers. When a tag was detected, the $SE(3)$ transform instantly corrected the robot's pose estimation.

Discussion: This hybrid localization approach proved vastly superior for this specific hardware platform. Attempting to run full ORB-SLAM2 would have monopolized the remote workstation's resources and introduced high latency. By trading the theoretical elegance of SLAM for the practical reliability of fiducial marker correction, the system achieved the necessary spatial accuracy to navigate a "Live City" without compromising the 10ms control loop. This demonstrates a vital engineering principle: leveraging environmental structure (markers) to offset hardware deficiencies.

Human-Machine Interface (HMI) Use-Case Demo

The cross-platform Web/Mobile application served as the primary demonstration of the full system integration. During use-case testing, the application successfully discovered the active Duckiebot on the LAN via mDNS and established a WebSocket connection.

The 3D environment, rendered via Three.js, accurately visualized the live telemetry of the robot in real-time. Operators could visually track the robot's pose along the map, monitor the live YOLOv5 bounding boxes, and observe the projected DWA trajectories.

Discussion: The HMI acts as more than just a visualizer; it represents a functional control tower. The ability to seamlessly switch between full autonomy (issuing coordinate commands to the A* planner) and manual override (using virtual joysticks) validates the robustness of the ROS node architecture. The system smoothly handled interruptions and overrides without crashing the underlying control loops, showcasing a highly fault-tolerant design suitable for fleet management.

Engineering Trade-offs and Scope Adjustments

Finally, the results of this project must be viewed through the lens of strategic scope reduction. During the first semester, it became evident that pursuing complex multi-agent "Convoy Formations" introduced massive coordination overhead.

Discussion: The decision to abandon swarm behaviors in favor of perfecting a highly robust, single-agent "Live City" baseline is a result in its own right. It represents a maturation in engineering methodology, shifting from a "feature-heavy" mindset to a "stability-first" paradigm. By focusing all computational and development resources on robust perception, distributed architecture, and hybrid planning, the final output is a highly reliable platform rather than a fragile, multi-featured prototype.

The Impact and Future Directions (/ 15 Points)

Explain the potential (or current if exist) impacts of your outcome in terms of how the methods and results will be used in real life, how it will change an existing process, or where it will be published, etc. Also, explain what would be the next step if the project is continued in the future, what kind of qualitative and/or quantitative updates can be made, shorty, where this project can go from here? This section should be between 250-500 words.

Project Impact

The outcomes of this project present significant impacts across educational, institutional, and industrial domains.

- **Educational and Institutional Impact:** As a culminating engineering project, the primary immediate impact is the comprehensive hands-on experience gained in integrating complex robotic systems, spanning sensor fusion, hybrid motion planning, and distributed networking. Institutionally, the complete, highly stable autonomous navigation stack delivered by this project will serve as a robust, open-source baseline platform. Future students at Hacettepe University can utilize this reliable foundation to explore more advanced artificial intelligence and robotics research without needing to rebuild foundational control loops.
- **Broader Real-World Application:** Beyond the Duckietown testbed, the architectural principles established here have direct real-world implications. By proving that computationally heavy tasks (like YOLOv5s perception and DWA planning) can be effectively offloaded from edge devices via a LAN-optimized ROS network, this methodology can be directly applied to industrial scenarios. For example, differential-drive robots used in warehouse logistics could utilize this exact distributed architecture to achieve high-level autonomy while keeping onboard hardware costs

Future Directions

While the current single-agent navigation stack is highly robust and effectively handles core tasks like lane following and dynamic obstacle avoidance, several qualitative updates can be pursued to evolve the system further:

- **"Live City" Concept and Rule-Based Autonomy:** The current architecture can be expanded to fully realize a "Live City" environment. Future iterations should focus on translating visual markers (such as AprilTags) and YOLO-based detections into high-level behavioral logic for traffic rule compliance. This includes stopping at red lights, yielding at complex intersections, and obeying directional traffic signs to achieve true urban autonomy.
- **Autonomous Parking Capabilities:** Building upon the existing hybrid path planner and dynamic obstacle memory, the system can be upgraded to perform complex parking maneuvers. By utilizing the global cost-map and precise local trajectory simulations from the DWA, the Duckiebot could identify vacant parking spots and autonomously execute parallel or perpendicular parking within the structured environment.

- **Multi-Robot Coordination and Swarm Logic:** With the single-agent baseline now stabilized, future development can confidently reintroduce the multi-robot coordination goals that were initially descoped. The established LAN-optimized communication stack can be expanded to broadcast position and velocity data between multiple Duckiebots. This will facilitate advanced vehicle following and cooperative multi-agent swarm behaviors, significantly expanding the scope of the platform.